1 Cloud Files — Project Report

Date: 20 October 2026

Deployment: https://cloudfiles-8rctkv2h.manus.space

Checkpoint: `5a017658b168e965bc1f53c054d1d2667b1ad6d6`

1.1 Aim

The aim of Cloud Files is to provide a small, secure, full-stack cloud file-storage application similar to a focused Google Drive. Each user can create an email/password account, upload common files up to 50 MB, browse only their own metadata, search and sort their file register, download through temporary signed URLs, remove files from the app index, and see storage usage.

1.2 Procedure

1. **Project creation:** Initialized a managed Webdev React/Express/tRPC/Drizzle project with server and database capabilities enabled. The managed platform's private S3-backed object storage was selected as the cloud-object-storage equivalent, so no AWS access keys were required.
2. **Database design:** Extended the users table with local password hashes and added a files table containing file ID, owner ID, file name, size, MIME type, storage key, status, upload timestamp, and deletion timestamp. The additive migration was generated and applied with Drizzle.
3. **Authentication:** Implemented email/password sign-up, login, logout, salted `scryptSync` password hashing, and the starter-compatible signed session cookie. Protected procedures resolve the session to a database user before operating on files.
4. **Direct upload:** The backend validates the file size and creates a server-owned storage key. It requests a temporary presigned PUT URL, returns that URL to the browser, and the browser uploads directly to private storage using XMLHttpRequest so progress can be displayed. Metadata is finalized only after the direct PUT succeeds.
5. **Secure download and deletion:** A download URL is issued only after a lookup constrained by both file ID and authenticated owner ID. Delete is owner-scoped and hides the metadata row and usage from the application index. The managed runtime currently does not expose a runtime object-delete operation, so this is documented as a soft delete.
6. **Dashboard:** Built a responsive dashboard with a calm editorial utility style: drag-and-drop upload surface, progress bar, file register, search, name/date sorting, storage usage card, download action, delete confirmation, and clear toast feedback.
7. **Validation:** Ran `pnpm check` and `pnpm build`; verified `/api/health`; created a real account in the preview; verified session-backed file listing; issued a signed upload URL; completed a direct storage PUT with HTTP 200; and downloaded the resulting object through an owner-checked signed URL with HTTP 200 and matching bytes.
8. **Deployment:** Committed and pushed the completed project to the canonical `main` branch and published checkpoint `5a017658b168e965bc1f53c054d1d2667b1ad6d6` to the public URL above.

1.3 Architecture

The browser communicates with the Express/tRPC backend for authentication, metadata, ownership checks, and temporary signed URL issuance. File bytes move directly between the browser and private

object storage. The backend stores only metadata and never proxies the file contents. The full editable Mermaid source is included with the project at `docs/architecture.mmd`.

1.4 Dashboard screenshot

The following screenshot was captured after a real account was created and authenticated in the Cloud Files dashboard. It shows the workspace header, upload surface, private storage card, search/sort controls, empty file register, and signed-transfer security note.

1.5 Result

Cloud Files is deployed and reachable online at **https://cloudfiles-8rctkv2h.manus.space**. The repository includes the complete source, migration, README, architecture diagram, and environment-variable guidance. The managed deployment uses platform-provided `DATABASE_URL`, `MANUS_API_URL`, `MANUS_API_KEY`, `MANUS_JWT_SECRET`, `MANUS_PROJECT_ID`, and `PORT` values; secrets are not hardcoded into the browser bundle or source files.

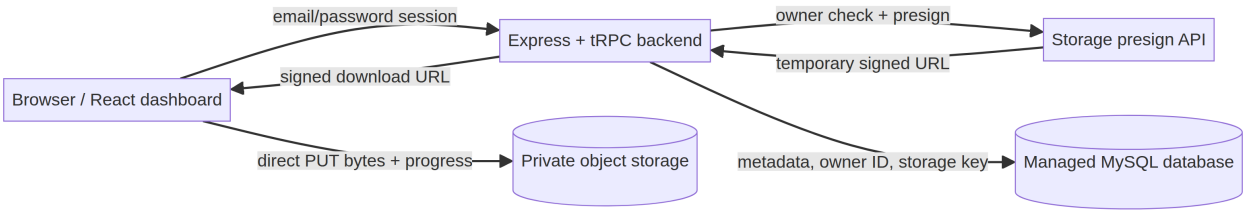


图 1　Cloud Files architecture

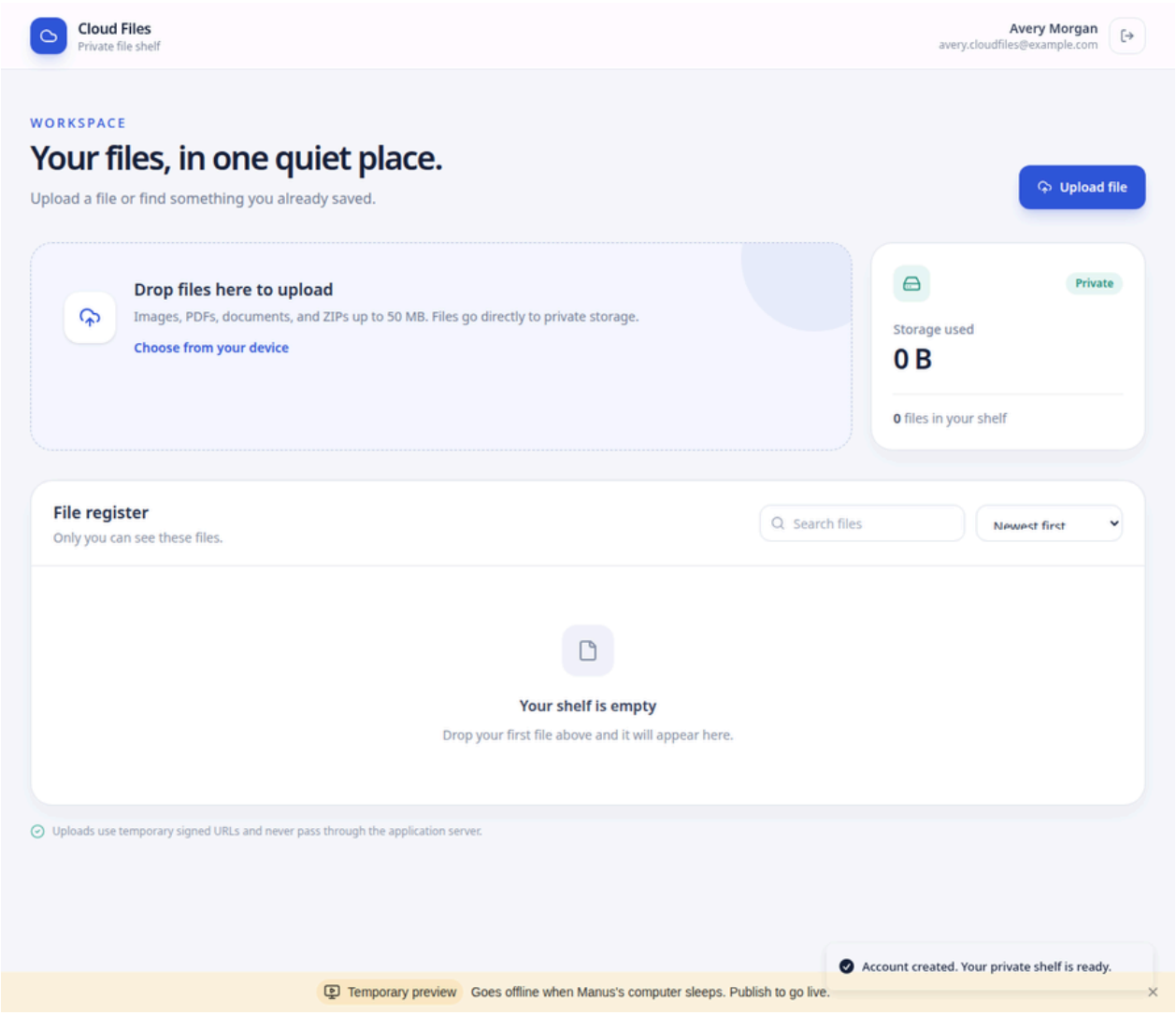


图 2　Authenticated Cloud Files dashboard captured during validation